



SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

FAKULTET PRIMIJENJENE MATEMATIKE I INFORMATIKE

Sveučilišni prijediplomski studij Matematika i računarstvo

Sustav za elektroničko glasanje

ZAVRŠNI RAD

Mentor:

dr. sc. Jurica Maltar

Kandidat:

Ivan Vuletić

Osijek, 2026.

Claude korišten je kao pomoć pri pisanju ovog rada.

- Generativna tehnologija korištena je za ispravke, poboljšanje stila i prikupljanje ideja.
- Nijedan generirani sadržaj činjenične prirode ili vezan uz temu rada nije predstavljen kao izvorno autorovo djelo.
- Autor preuzima potpunu odgovornost za cjelokupni sadržaj ovog rada.

Ova izjava sastavljena je u skladu s *Okvirom korištenja alata umjetne inteligencije na Fakultetu primijenjene matematike i informatike Sveučilišta Josipa Jurja Strossmayera u Osijeku*.

Sadržaj

1	Uvod	1
2	Arhitektura sustava	3
2.1	Poslužiteljska aplikacija	3
2.2	Admin konzola i glasački terminal	4
3	WebSocket komunikacija u stvarnom vremenu	5
3.1	Protokol i mehanizam rada	5
3.2	Održavanje veze - ping/pong okviri	6
3.3	Dva odvojena kanala	6
3.4	Emitiranje događaja tek nakon spremanja	7
4	Sigurnost glasanja	9
4.1	Asimetrična kriptografija	9
4.2	Model “zapečaćene kutije”	9
4.3	Hibridna shema enkripcije	10
4.4	Jednokratni tokeni i sprječavanje dvostrukog glasanja	11
4.4.1	Životni ciklus tokena	11
4.4.2	Problem trke (race condition)	11
5	Integritet podataka i revizijski trag	13
5.1	Integritet podataka	13
5.2	Revizijski trag	14
6	Implementacija	15
6.1	Autentifikacija i autorizacija	15
6.1.1	Administratorska autentifikacija	15
6.1.2	Autentifikacija uređaja	16
6.2	Poslužiteljska aplikacija	16
6.3	Administratorska konzola	17
6.4	Glasački terminal	17
6.5	Tijek cijelog izbornog procesa	17
7	Zaključak	19
	Literatura	21

Sažetak	23
Summary	25
Životopis	27

1 | Uvod

Elektroničko glasanje sve je više prisutno kao dopuna ili zamjena tradicionalnim oblicima glasovanja. Zemlje poput Estonije koriste sustave internetskog glasanja na nacionalnoj razini već od 2005. godine, dok u jednoj od popularnijih demokracija - SAD-u oblici elektroničkog glasanja predstavljeni su već u devedesetim godinama prošlog stoljeća. U Hrvatskoj takav sustav još nije populariziran, no neke političke opcije koriste ga u unutarstranačkim izborima. Prema podacima državnog izbornog povjerenstva "super" izborna 2024. godina porezne obveznike koštala je okvirno 39 milijuna eura, što ističe najveću prednost elektroničkog glasanja - smanjenje troškova. Uz sve tehničke zahtjeve sustav mora imati i povjerenje korisnika. Birač mora vjerovati da je njegov glas točno zabilježen, da ga nitko ne može izmijeniti, ali najvažnije da se glas ne može naknadno povezati s njegovim identitetom.

Cilj ovog završnog rada je razvoj sustava za elektroničko glasanje koji te zahtjeve rješava na razini arhitekture i implementacije, s posebnim naglaskom na WebSocket komunikaciju u stvarnom vremenu. Sustav se sastoji od tri međusobno povezane cjeline: poslužiteljske aplikacije koja upravlja izborima, biračkim mjestima, tokenima i glasovima; administratorske konzole za nadzor izbornog procesa u stvarnom vremenu putem WebSocket veze; te glasačkog terminala namijenjenog biraču na biralištu. Poslužiteljska aplikacija i administratorska konzola povezane su trajnom, dvosmjernom WebSocket vezom koja administratoru omogućuje uvid u tijek glasanja bez ikakvog kašnjenja ili ručnog osvježavanja, osiguravajući da se privatnost glasa i sigurnost rezultata ne oslanjaju na povjerenje u administratora, već su ugrađeni u samu arhitekturu i koncept glasanja: glas se šifrira na biralištu i može se dešifrirati tek zajedničkim djelovanjem svih glasova nakon zatvaranja izbora, a tokeni za glasanje ne nose nikakav trag identiteta birača, te svaka izmjena podataka ostavlja trag koji se može neovisno provjeriti.

Rad je strukturiran na sljedeći način. Drugo poglavlje opisuje arhitekturu sustava i način na koji tri navedene cjeline komuniciraju. Treće poglavlje, središnje za ovaj rad, detaljno opisuje komunikaciju u stvarnom vremenu putem WebSocket protokola. Četvrto i peto poglavlje nadovezuju se na to opisujući sigurnosne mehanizme koji štite podatke što se tom komunikacijom razmjenjuju: asimetričnu enkripciju i tajnost glasa, jednokratne tokene i sprječavanje dvostrukog glasanja, integritet podataka te revizijski trag. Šesto poglavlje opisuje implementaciju pojedinih dijelova sustava te cjelokupan tijek jednog izbornog procesa.

2 | Arhitektura sustava

Sustav je izgrađen prema klijent-poslužitelj arhitekturi, u kojoj poslužiteljska - admin konzola jedina prikazuje stanje izbora u realnom vremenu, dok su klijentske aplikacije odgovorne samo za prikaz podataka i proslijeđivanje zahtjeva korisnika. Komunikacija između klijenata i poslužitelja odvija se putem REST API-ja - arhitekturnog stila u kojem se svaki resurs (izbor, biralište, uređaj, token, glas) adresira jedinstvenom putanjom, a nad njim se izvode standardne HTTP metode (GET za dohvat, POST za stvaranje, PATCH za izmjenu).

2.1 Poslužiteljska aplikacija

Poslužiteljska aplikacija razvijena je u okviru *FastAPI*, asinkronom web okviru za Python koji automatski generira i validira sheme zahtjeva i odgovora pomoću biblioteke *Pydantic* [1]. Podaci se trajno pohranjuju u relacijskoj bazi *PostgreSQL*, kojoj se pristupa preko *SQLAlchemy* objektno-relacijskog preslikavanja (ORM), a promjene sheme baze prate se alatom *Alembic* [2].

API je organiziran u logičke module (*router*e), svaki odgovoran za jednu domenu sustava: prijavu administratora, izbore i kandidate, biračka mjesta, uređaje, tokene, glasanje, WebSocket komunikaciju te revizijski trag. Svaki router uvozi se i registrira u ulaznoj točki aplikacije:

```
1 app.include_router(auth_router)
2 app.include_router(elections_router)
3 app.include_router(stations_router)
4 app.include_router(devices_router)
5 app.include_router(tokens_admin_router)
6 app.include_router(voting_router)
7 app.include_router(websocket_router)
8 app.include_router(audit_router)
```

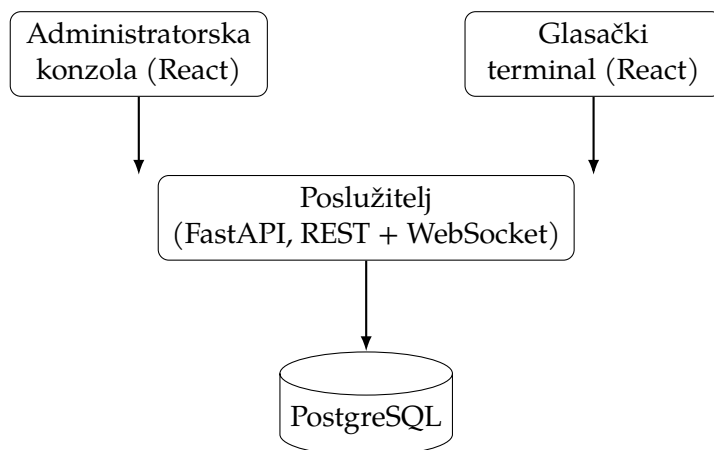
Listing 2.1: Registracija routera u `main.py`.

Ovakva podjela drži kod odvojen po odgovornosti (svaki modul zna samo za svoju domenu) i olakšava primjenu različitih pravila autorizacije po skupini ruta - primjerice, rute za upravljanje tokenima podijeljene su na `admin_router` (zaštićen administratorskim JSON Web Tokenom - JWT) i `device_router` (zaštićen API ključem uređaja), iako obje vrše radnje nad istim podacima.

2.2 Admin konzola i glasački terminal

Obje klijentske aplikacije, administratorska konzola i glasački terminal, izrađene su kao jednostranične aplikacije (engl. *Single Page Application*, SPA) korištenjem *React* biblioteke [3], čime se korisničko sučelje dinamički ažurira bez ponovnog učitavanja stranice, primjerice pri primitku WebSocket događaja u stvarnom vremenu. Administratorska konzola omogućuje stvaranje i zatvaranje izbora, upravljanje kandidatima, biračkim mjestima, glasačkim uređajima i tokenima, te nadzor izbornog procesa u stvarnom vremenu i prebrojavanje glasova. Glasački terminal je aplikacija samo s jednim zadatkom: provesti birača kroz proces unosa tokena, odabira kandidata, te potvrdu ili promjenu glasa, bez mogućnosti pristupa bilo čemu izvan tog toka.

Razdvajanje u dvije potpuno odvojene aplikacije, umjesto jedne aplikacije s dvije uloge, tehničko - sigurnosna je odluka - glasački terminal fizički stoji na biralištu i dostupan je javnosti, pa ne smije ni u kojem obliku sadržavati kod ili sučelje namijenjeno administratoru, jer ne samo da konzola sadrži upravljačke mogućnosti, već i stvarne podatke o glasanju koji ne smiju biti javno dostupni dok se izbori ne zaključaju.



Slika 2.1: Pojednostavljen prikaz arhitekture sustava.

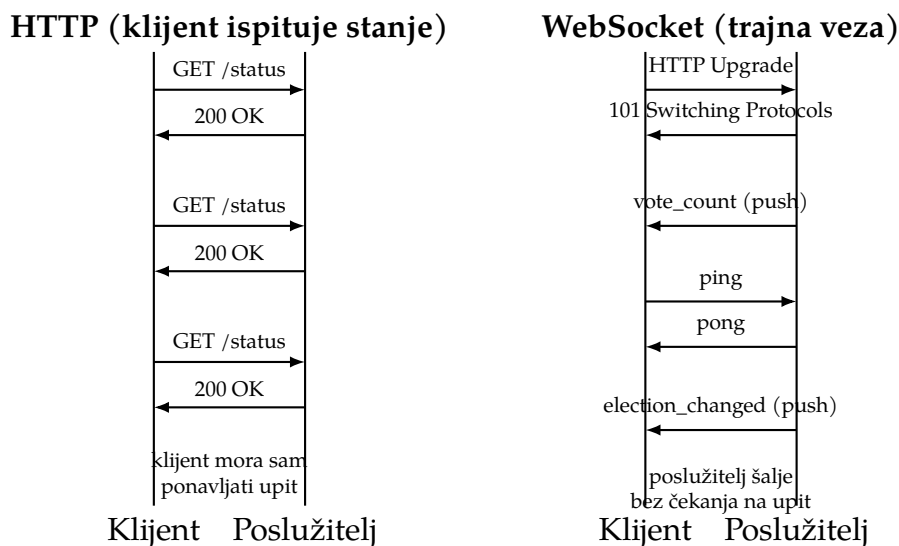
3 | WebSocket komunikacija u stvarnom vremenu

HTTP protokol je po prirodi zahtjev-odgovor: klijent mora sam započeti svaki zahtjev, poslužitelj ne može klijentu ništa poslati bez da klijent to prvo ne zatraži. Za administratorsku konzolu to ne odgovara jer nadzorni zaslon mora prikazivati broj pristiglih glasova uživo, bez da se korisnik ručno osvježava. Rješenje je *WebSocket* protokol, koji nakon početnog HTTP rukovanja uspostavlja trajnu, dvosmjernu vezu preko koje obje strane u bilo kojem trenutku mogu poslati poruku drugoj [4].

3.1 Protokol i mehanizam rada

WebSocket veza započinje kao običan HTTP zahtjev koji sadrži posebna zaglavlja kojima klijent traži unaprjeđenje veze: `Upgrade: websocket` i `Connection: Upgrade`, uz nasumično generiran ključ u zaglavlju `Sec-WebSocket-Key`. Ako poslužitelj podržava WebSocket, odgovara statusnim kodom 101 `Switching Protocols` i zaglavljem `Sec-WebSocket-Accept`, izračunatim iz poslanog ključa - čime potvrđuje da razumije zahtjev za nadogradnju, a ne da je riječ o slučajno sličnom HTTP zahtjevu [4]. Nakon ove jedne razmjene, isti mrežni (TCP) spoj koji je do tada nosio HTTP promet nastavlja postojati, ali se njime dalje razmjenjuju WebSocket *okviri* (frames), a ne HTTP poruke - veza ostaje otvorena sve dok je jedna od strana izričito ne zatvori.

Ključna razlika prema HTTP-u nije samo to što veza ostaje otvorena, već da obje strane mogu inicirati slanje poruke u bilo kojem trenutku. Kod HTTP-a klijent mora sam ponoviti zahtjev da bi saznao je li se nešto promijenilo (tzv. *polling*); kod WebSocket veze poslužitelj poruku šalje čim nastupi događaj, bez da je itko to od njega zatražio (slika 3.1). HTTP je prikladan za dohvat podataka koji se rijetko mijenjaju, dok za izbore koji se prate u stvarnom vremenu bolje rješenje je WebSocket, jer omogućuje da se administratorska konzola odmah osvježi bez da korisnik ručno traži nove podatke. Razlika između ta dva pristupa slična je razlici između razmjene poruka i telefonskog poziva: svaka HTTP poruka zaseban je, neovisan prijenos koji netko mora sam pokrenuti, dok jednom uspostavljena WebSocket veza, poput telefonskog poziva, omogućuje trenutačnu razmjenu u oba smjera bez potrebe da se za svaku novu informaciju veza iznova uspostavlja.



Slika 3.1: Usporedba komunikacijskog obrasca: HTTP ispitivanje stanja (*polling*) nasuprot trajnoj WebSocket vezi s heartbeat mehanizmom (ping/pong).

3.2 Održavanje veze - ping/pong okviri

Kako WebSocket veza može ostati otvorena satima bez da se njome išta pošalje, protokol definira posebne kontrole, ping i pong, isključivo u svrhu provjere je li druga strana i dalje prisutna. Bilo koja strana smije u bilo kojem trenutku poslati ping okvir; strana koja ga primi *mora* odgovoriti pong okvirom s identičnim sadržajem, i to bez sudjelovanja aplikacijskog koda - provjera se odvija na razini same WebSocket implementacije [4]. Ovaj mehanizam, uobičajeno nazivan *heartbeat*, služi dvjema svrhama: otkriva prekinutu vezu (primjerice, birač isključi terminal iz struje usred sesije) puno prije nego što bi to učinio standardni TCP mehanizam za istek veze, i sprječava da posrednički mrežni uređaji (proxy poslužitelji) sami zatvore naizgled neaktivnu vezu. U poslužiteljskoj aplikaciji ovaj mehanizam ne implementira se ručno u aplikacijskom kodu, već ga u pozadini provodi ugrađena WebSocket implementacija unutar ASGI poslužitelja (*uvicorn*) na kojem FastAPI radi.

3.3 Dva odvojena kanala

Sustav izlaže dva WebSocket krajnja mjesta s različitom namjenom i različitim skupom primatelja poruka:

- `/ws/admin` - koristi ga administratorska konzola. Zahtijeva autentifikaciju: JWT token šalje se kao *prva poruka nakon* uspostave veze, nikad kao dio URL-a, jer URL-ovi (za razliku od tijela poruke) mogu završavati zapisani u logovima poslužitelja i posredničkih proxy poslužitelja. Ovaj kanal prima sve događaje, uključujući brojeve glasova.

- `/ws/status` - koristi ga glasački terminal. Nema autentifikacije, jer poruke koje njime prolaze ne smiju sadržavati ništa osjetljivo - terminal njime prima isključivo signal da se nešto promijenilo (npr. da je izbor otvoren ili zatvoren), nakon čega stvarne podatke sam ponovno dohvaća REST pozivom.

```

1 def broadcast_admin(self, message: dict) -> None:
2     """Samo admin dashboardu - npr. vote_count."""
3     if self._loop is None or not self.admin_connections:
4         return
5     asyncio.run_coroutine_threadsafe(self._broadcast_admin_async(
6         message), self._loop)
7
8 def broadcast_all(self, message: dict) -> None:
9     """Admin dashboardu I glasackim terminalima - SAMO za evente
10    bez
11    ikakvog osjetljivog sadrzaja, npr. election_changed."""
12    if self._loop is None or not (self.admin_connections or self.
13        status_connections):
14        return
15    asyncio.run_coroutine_threadsafe(self._broadcast_all_async(
16        message), self._loop)

```

Listing 3.1: Sadržajno razdvajanje dvaju kanala u `ConnectionManager`-u.

3.4 Emitiranje događaja tek nakon spremanja

FastAPI podržava i sinkrone i asinkrone rute. Ruta koja zaprima glas (`POST /voting/cast`) je sinkrona FastAPI ruta - radi korak po korak (provjeri token, spremi glas, potvrdi transakciju), dok je slanje WebSocket poruka asinkrona operacija - jer poslužitelj mora odjednom komunicirati s više otvorenih veza (glasačkih mjesta) i slati poruke čim nešto primi, umjesto da svaku vezu, odnosno zahtjev riješi jedan po jedan. Zato WebSocket funkcionira u tzv. asinkronoj petlji koja upravlja tim operacijama (svim WebSocket vezama, ping/pong okvirima, itd.). Problem nastaje kada zaprimamo glas i spremamo ga u bazu, to se odvija u tzv. sinkronoj ruti koja nije u toj asinkronoj petlji, a treba poslati WebSocket poruku administratoru koja se nalazi na drugoj ruti - asinkronoj. Rješenje je funkcija `asyncio.run_coroutine_threadsafe` koja na siguran način spaja sinkronu rutu sa asinkronom petljom za izvršavanje WebSocket zahtjeva.

Važno je da se WebSocket poruka šalje *tek nakon* što je transakcija nad bazom uspješno potvrđena (`commit`). Kad bi se poruka poslala prije potvrde, a transakcija zatim propala, administratorska konzola prikazala bi glas koji zapravo nikad nije zabilježen. Sadržaj poruke je s razlogom sveden na najmanju moguću mjeru - tip događaja, identifikator izbora i biračkog mjesta te agregirani broj glasova - i nikad ne sadrži token, identifikator kandidata niti sadržaj glasa:

```

1 db.commit()

```

```
2 db.refresh(vote)
3
4 vote_count = (
5     db.query(func.count(Vote.id))
6     .filter(Vote.station_id == station_id, Vote.election_id ==
7         election.id)
8     .scalar()
9 )
10 broadcast_event({
11     "type": "vote_count",
12     "election_id": str(election.id),
13     "station_id": str(device.station_id),
14     "count": vote_count,
15 })
```

Listing 3.2: Emitiranje događaja nakon uspješnog spremanja glasa.

4 | Sigurnost glasanja

Temeljni sigurnosni zahtjev sustava je da nitko - pa ni administrator izbora - ne može saznati kako je pojedini birač glasao prije nego što je izbor zatvoren i svi glasovi zajedno prebrojani, a da pritom nijedan birač ne može glasati više od jednom. Prvi dio ovog poglavlja opisuje kako se to postiže enkripcijom svakog glasa u trenutku glasanja, pri čemu je ključ za dešifriranje odvojen od sustava sve do trenutka prebrojavanja; drugi dio opisuje mehanizam jednokratnih tokena kojim se sprječava dvostruko glasanje, te potpuno eliminira mogućnost povezivanja identiteta birača s njegovim glasom.

4.1 Asimetrična kriptografija

Kod simetrične enkripcije isti se ključ koristi i za šifriranje i za dešifriranje, što znači da bi taj ključ morao postojati na biralištu u trenutku glasanja - a time bi i dešifriranje bilo moguće odmah, čim postoji prvi glas. Asimetrična (javno-ključna) kriptografija rješava taj problem parom matematički povezanih ključeva: javnim ključem, koji se koristi isključivo za šifriranje, i privatnim ključem, koji je jedini sposoban to šifriranje poništiti. Sustav koristi RSA algoritam s ključem duljine 2048 bita.

4.2 Model “zapečaćene kutije”

Pri otvaranju izbora poslužitelj generira novi par RSA ključeva. Javni ključ sprema se u bazu podataka uz taj izbor i koristi se za enkripciju svakog glasa. Privatni ključ se *vraća administratoru samo jednom*, u tom istom trenutku, i nikad se ne sprema u sustav - ni u bazu podataka, ni u zapisnike rada aplikacije:

```
1 def generate_election_keypair() -> tuple[str, str]:
2     """Vraca (public_key_pem, private_key_pem)."""
3     private_key = rsa.generate_private_key(public_exponent=65537,
4     key_size=2048)
5     public_key = private_key.public_key()
6     private_pem = private_key.private_bytes(
7         encoding=serialization.Encoding.PEM,
8         format=serialization.PrivateFormat.PKCS8,
9         encryption_algorithm=serialization.NoEncryption(),
10    ).decode("utf-8")
11    public_pem = public_key.public_bytes(
```

```

11         encoding=serialization.Encoding.PEM,
12         format=serialization.PublicFormat.SubjectPublicKeyInfo,
13     ).decode("utf-8")
14     return public_pem, private_pem

```

Listing 4.1: Generiranje para ključeva izbora (elections/crypto.py).

Dizajniranjem ovakvog sustava osigurava se maksimalna tajnost i otpornost na brute-force napade, ali zato ako administrator izgubi privatni ključ rezultati izbora neće se moći prebrojati ni na koji način, ni od strane razvijatelja sustava, ni izravnim pristupom bazi podataka, jer ključ doslovno nigdje ne postoji osim u onom trenutku kad ga je administrator preuzeo. To zvuči malo nepromišljeno s obzirom da je u pitanju i mogućnost ljudske greške, ali to i je bio cilj jer tajnost glasa ne smije ovisiti o tome vjeruje li se osobi koja sustav održava.

4.3 Hibridna shema enkripcije

Čisti RSA nije prikladan za šifriranje proizvoljno velikog sadržaja - ključ duljine 2048 bita može šifrirati poruku ograničene duljine - što kraće to bolje. Zato sustav koristi hibridnu - dvoslojnu shemu: sadržaj glasa šifrira se simetričnim AES-256-GCM ključem koji se generira nasumično za svaki pojedini glas, a taj (kratki) AES ključ zatim se šifrira RSA-OAEP shemom javnim ključem izbora [6]. GCM način rada dodatno pruža autentikaciju sadržaja - svaki pokušaj izmjene šifrata nakon enkripcije bit će otkriven pri dešifriranju [7].

```

1 def encrypt_vote(public_key_pem: str, candidate_id: uuid.UUID) ->
   str:
2     public_key = serialization.load_pem_public_key(public_key_pem.
   encode("utf-8"))
3     plaintext = json.dumps({
4         "candidate_id": str(candidate_id),
5         "nonce": base64.b64encode(os.urandom(16)).decode("ascii"),
6     }).encode("utf-8")
7
8     aes_key = AESGCM.generate_key(bit_length=256)
9     iv = os.urandom(12)
10    ciphertext = AESGCM(aes_key).encrypt(iv, plaintext, None)
11    encrypted_key = public_key.encrypt(aes_key, _OAEP_PADDING)
12
13    return ".".join([
14        base64.b64encode(encrypted_key).decode("ascii"),
15        base64.b64encode(iv).decode("ascii"),
16        base64.b64encode(ciphertext).decode("ascii"),
17    ])

```

Listing 4.2: Enkripcija glasa (voting/crypto.py).

Nasumičan nonce unutar sadržaja poruke dodatna je mjera opreza jer bez njega bi dva glasa za istog kandidata, u teoriji, mogla proizvesti prepoznatljivo

sličan uzorak prije enkripcije. Dešifriranje (`decrypt_vote`) koristi se samo u Admin konzoli, u dijelu nakon zaključavanja izbora, u Nadzor kartici za prebrojavanje glasova. Nakon što administrator ručno unese privatni ključ - taj se ključ tada drži samo u memoriji preglednika/poslužitelja za vrijeme trajanja izračuna i nigdje se ne pohranjuje, te se koristi za dešifriranje svakog glasa i prebrojavanje glasova po kandidatima.

4.4 Jednokratni tokeni i sprječavanje dvostrukog glasanja

4.4.1 Životni ciklus tokena

Prije glasanja, administrator generira skup tokena po biralištu - po jedan za svakog očekivanog birača. Token je 128-bitni nasumičan niz, kodiran u Base32 abecedi (bez znamenki 0/1, radi lakše čitljivosti pri fizičkoj podjeli birača) i raspoređen u blokove od 5 znakova. Baza pohranjuje isključivo HMAC-SHA256 hash tokena, nikad sam token:

```
1 def generate_raw_token() -> str:
2     raw_bytes = secrets.token_bytes(16) # 128 bit
3     encoded = base64.b32encode(raw_bytes).decode("ascii").rstrip("=")
4     grouped = "-".join(encoded[i : i + 5] for i in range(0, len(encoded), 5))
5     return grouped
6
7 def hash_token(raw_token: str) -> str:
8     normalized = normalize_token(raw_token)
9     return hmac.new(TOKEN_HASH_PEPPER.encode(), normalized.encode(),
10                    hashlib.sha256).hexdigest()
```

Listing 4.3: Generiranje i hashiranje tokena (`tokens/crypto.py`).

Token ima točno dva moguća stanja, `AVAILABLE` i `USED`, i prijelaz je dopušten isključivo u jednom smjeru. Nijedna tablica u sustavu ne povezuje token s identitetom birača niti token s konkretnim glasom - ta veza uopće ne postoji ni u jednom trenutku, u bilo kojem obliku.

4.4.2 Problem trke (race condition)

Kad bi provjera stanja tokena i njegovo označavanje kao iskorištenog bile dvije odvojene operacije - prvo `SELECT` da se provjeri je li token dostupan, zatim, ako jest, `UPDATE` koji ga označava iskorištenim - nastao bi klasičan problem trke - *check-then-act*. Dva zahtjeva s istim tokenom, poslana gotovo istovremeno, mogla bi oba proći provjeru prije nego što bilo koji od njih stigne izvršiti izmjenu, i oba bi glasa bila prihvaćena istim tokenom.

12 4.4. JEDNOKRATNI TOKENI I SPRJEČAVANJE DVOSTRUKOG GLASANJA

Rješenje je da se provjera i potrošnja tokena izvedu kao *jedna jedinstvena operacija* na razini retka baze podataka, korištenjem uvjetovanog UPDATE ... WHERE ... RETURNING izraza:

```
1 result = db.execute(  
2     update(VoteToken)  
3     .where(  
4         VoteToken.token_hash == token_hash,  
5         VoteToken.status == TokenStatus.AVAILABLE,  
6         VoteToken.station_id == device.station_id,  
7     )  
8     .values(status=TokenStatus.USED)  
9     .returning(VoteToken.id, VoteToken.station_id)  
10 )  
11 row = result.first()
```

Listing 4.4: Atomarna potrošnja tokena (voting/router.py).

Baza podataka osigurava da, kad dva paralelna zahtjeva pokušaju izvršiti ovaj UPDATE nad istim retkom, samo jedan od njih može uspjeti pronaći redak čiji je status još uvijek AVAILABLE - onaj koji ga prvi zaključa. Drugi zahtjev, iako pokrenut u istoj milisekundi, na svom WHERE uvjetu neće naći nijedan redak (jer je status u međuvremenu već promijenjen) i row će biti None, što sustav prepoznaje kao pokušaj ponovne uporabe već iskorištenog tokena. Cijela operacija - potrošnja tokena, enkripcija glasa i njegovo spremanje - odvija se unutar jedne transakcije nad bazom podataka, pa u slučaju bilo kakve pogreške nijedan dio ne ostaje zapisan bez ostalih.

5 | Integritet podataka i revizijski trag

Uz tajnost glasa i sprječavanje dvostrukog glasanja, sustav mora omogućiti i provjeru da podatci nakon spremanja nisu naknadno izmijenjeni, te administratoru dati uvid u sigurnosno važne događaje. Prvi dio ovog poglavlja opisuje lančani hash kojim se otkriva naknadna izmjena podataka, a drugi dio revizijski trag sistemskih i sigurnosnih događaja.

5.1 Integritet podataka

Enkripcija glasa štiti njegovu tajnost, ali sama po sebi ne jamči da netko nakon spremanja neće promijeniti ili obrisati neki od zapis u bazi - takva izmjena šifriranog teksta ne bi bila vidljiva sve do dešifriranja, a ako je riječ o brisanju retka, ne bi se vidjela ni tada. Taj problem riješen je lančanim hash-om, sigurnosno sličnim onome koji koriste blockchain sustavi, gdje svaki blok također sadrži hash prethodnog bloka: svaki glas u sebi nosi SHA-256 hash izračunat nad vlastitim šifratom, hash-om *prethodnog* glasa u istom izboru i identifikatorom izbora.

```
1 def compute_integrity_hash(ciphertext_b64: str, prev_hash: str |  
    None, election_id: str) -> str:  
2     payload = f"{ciphertext_b64}|{prev_hash or ''}|{election_id}"  
3     return hashlib.sha256(payload.encode("utf-8")).hexdigest()  
4  
5 def verify_chain(votes: list[tuple[str, str | None, str]],  
    election_id: str) -> bool:  
6     expected_prev = None  
7     for encrypted_vote, prev_hash, integrity_hash in votes:  
8         if prev_hash != expected_prev:  
9             return False  
10        if compute_integrity_hash(encrypted_vote, prev_hash,  
            election_id) != integrity_hash:  
11            return False  
12        expected_prev = integrity_hash  
13    return True
```

Listing 5.1: Lančani hash (voting/integrity.py).

Budući da hash svakog glasa ovisi o hashu prethodnog, izmjena ili brisanje bilo kojeg zapisa u lancu “lomi” sve hashove koji dolaze poslije njega. Provjera cijelog lanca (*verify_chain*) izvodi se na Nadzor zaslonu u Admin konzoli, prije prikaza rezultata, nad podacima sortiranim kronološkim redoslijedom unosa - eventualno

miješanje redoslijeda radi anonimizacije prikaza rezultata provodi se tek *nakon* te provjere, nad zasebnom kopijom podataka, da se sam poredak provjere nikad ne poremeti.

5.2 Revizijski trag

Uz zaštitu samog glasa, sustav mora administratoru omogućiti uvid u sve promjene u sustavu; prijave administratora, otvaranje i zatvaranje izbora, pokušaje glasanja nevažećim ili već iskorištenim tokenom, te neovlaštene pokušaje pristupa administratorskim API rutama. Svaki takav događaj bilježi se u tablicu `audit_logs` putem jedinstvene funkcije `log_event`:

```
1 def log_event(  
2     db: Session,  
3     event_type: str,  
4     *,  
5     admin_user_id: uuid.UUID | None = None,  
6     station_id: uuid.UUID | None = None,  
7     device_id: uuid.UUID | None = None,  
8     metadata: dict | None = None,  
9 ) -> None:  
10     entry = AuditLog(  
11         event_type=event_type,  
12         admin_user_id=admin_user_id,  
13         station_id=station_id,  
14         device_id=device_id,  
15         event_metadata=metadata,  
16     )  
17     db.add(entry)  
18     # namjerno se NE commita ovdje - poziva se unutar iste  
19     # transakcije  
20     # kao i sama operacija koja se logira, commit radi pozivatelj
```

Listing 5.2: Bilježenje revizijskog događaja (`audit/service.py`).

Funkcija sama ne izvršava `commit` - zapis o događaju sprema se unutar iste transakcije kao i operacija koju opisuje (npr. prihvata glasa), pa zapis aktivnosti nikad ne postoji bez odgovarajuće stvarne promjene, i obrnuto. Polje `event_metadata` također sam strogo ograničio na površne informacije (npr. broj generiranih tokena, identifikator izbora) - nikad ne sadrži sirovi token, hash tokena, identitet birača niti sadržaj glasa, pa zapis aktivnosti sam po sebi ne otkriva nikakve osjetljive podatke. Administratorska konzola prikazuje ove zapise u kartici “Sigurnosni i sistemski događaji”, s pokušajima koji ukazuju na moguću zlouporabu - nevažećim i već iskorištenim tokenima.

6 | Implementacija

6.1 Autentifikacija i autorizacija

Sustav razlikuje dva potpuno odvojena identiteta koja se autentificiraju na različit način: administratora, koji je čovjek s interaktivnom sesijom, i uređaj (glasački terminal), koji je stroj bez korisničkog unosa lozinke pri svakom glasu.

6.1.1 Administratorska autentifikacija

Lozinka administratora nikad se ne sprema u čitljivom obliku, već kao Argon2id hash - pobjednik natjecanja Password Hashing Competition i algoritam preporučen za hashiranje lozinke upravo zbog otpornosti na napade grubom silom ubrzane grafičkim procesorima, jer je memorijski zahtjevan (memory-hard) [8]. Nakon uspješne prijave, poslužitelj izdaje JWT (JSON Web Token) potpisan HS256 algoritmom, s rokom valjanosti od 12 sati. Taj token administratorska konzola šalje u Authorization zaglavlju svakog sljedećeg zahtjeva [5].

```
1 _hasher = PasswordHasher()
2
3 def hash_password(raw_password: str) -> str:
4     return _hasher.hash(raw_password)
5
6 def create_access_token(admin_id: str, username: str) -> str:
7     now = datetime.now(timezone.utc)
8     payload = {
9         "sub": str(admin_id),
10        "username": username,
11        "iat": now,
12        "exp": now + timedelta(hours=JWT_EXPIRE_HOURS),
13    }
14    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)
```

Listing 6.1: Hashiranje lozinke i izdavanje JWT-a (auth/security.py).

Prilikom neuspjele prijave poslužitelj namjerno vraća identičnu poruku bez obzira radi li se o nepostojećem korisničkom imenu ili pogrešnoj lozinke - razlikovanje tih dvaju slučajeva otkrilo bi napadaču koja korisnička imena uopće postoje u sustavu.

6.1.2 Autentifikacija uređaja

Glasački terminal registrira se u “Admin konzoli” prije početka izbora i vezan je za glasačko mjesto, uređaj sam po sebi ne bilježi nikakve podatke o biračima samo izbor glasa nakon unosa tokena. Pri registraciji uređaja generira se nasumičan API ključ koji se prikazuje administratoru samo jednom, a u bazi se sprema isključivo njegov HMAC-SHA256 hash (isto načelo kao kod tokena za glasanje, odjeljak 4.4):

```
1 def generate_device_api_key() -> str:
2     return secrets.token_urlsafe(32)
3
4 def hash_device_key(raw_key: str) -> str:
5     return hmac.new(DVICE_KEY_PEPPER.encode(), raw_key.encode(),
6                     hashlib.sha256).hexdigest()
```

Listing 6.2: Hashiranje API ključa uređaja (devices/security.py).

Taj ključ terminal šalje u vlastitom zaglavlju (X-Device-Key) uz svaki zahtjev prema rutama koje su rezervirane za uređaje (provjera i potrošnja tokena, slanje glasa), dok su administratorske rute zaštićene JWT provjerom. Razdvajanje ta dva mehanizma znači da kompromitacija jednog glasačkog terminala ne izlaže administratorska ovlaštenja, i obrnuto.

6.2 Poslužiteljska aplikacija

Poslužiteljska aplikacija implementirana je kroz module opisane u poglavlju 2. Sva-ka poznata pogreška u sustavu ima logičan, nazvan kod pogreške (npr. INVALID_TOKEN, TOKEN_ALREADY_USED, ELECTION_NOT_OPEN) koji se šalje klijentu - korisnik nikad ne vidi doslovnu grešku programskog jezika niti detalje o strukturi baze podataka. Sve neočekivane pogreške hvataju se funkcijom koja vrati generičku bezopasnu poruku, a pravi, detaljni zapis greške zapiše u interni log namijenjen za developere, čime se sprječava nenamjerno curenje tehničkih detalja napadaču koji ispituje sustav:

```
1 @app.exception_handler(Exception)
2 async def unhandled_error_handler(request: Request, exc: Exception)
3     :
4     logger.error("UNHANDLED_ERROR path=%s error_type=%s", request.
5     url.path, type(exc).__name__)
6     return JSONResponse(
7     status_code=500,
8     content={"error_code": "SERVER_ERROR", "message": "Doslo je
9     do neocekivane greske."},
10    )
```

Listing 6.3: Globalno rukovanje neočekivanim pogreškama (main.py).

Struktura baze prati glavno načelo očuvanja tajnosti izbora: primjerice, ne postoji tablica birača niti bilo koje polje koje bi token vezalo uz osobu ili glas uz token -

ta razdvojenost provedena je na cijeloj bazi čime je povezivanje identiteta i glasa dodatno svedeno na minimum i na razini izravnih upita nad bazom.

6.3 Administratorska konzola

Administratorska konzola organizirana je u nekoliko kartica: “Izbori i kandidati” za upravljanje životnim ciklusom izbora, “Biračka mjesta” s popisom svih 129 službenih birališta, “Uređaji i tokeni” za registraciju uređaja i generiranje bazena tokena po biralištu, te “Nadzor”, koja sadrži i uvid u tijek glasanja uživo (broj glasova po biralištu i po uređaju, revizijski trag) i, nakon zatvaranja izbora, prebrojavanje rezultata.

6.4 Glasački terminal

Glasački terminal provodi birača kroz jednostavan tijek: unos tokena, provjera valjanosti, odabir kandidata, potvrda, i konačno poruka o zaprimljenom glasu. Nakon svakog glasanja, kao i nakon dužeg razdoblja neaktivnosti, sučelje se automatski vraća na početni zaslon - sljedeći birač tako ne smije ni slučajno naslijediti nedovršenu sesiju prethodnog birača na istom uređaju. API ključ uređaja nikad se ne prikazuje na zaslonu niti je vidljiv biraču; koristi se isključivo u pozadinskim HTTP pozivima.

6.5 Tijek cijelog izbornog procesa

Radi cjelovitog pregleda, ovdje je opisan kompletan scenarij provedbe jednog izbora, od stvaranja do prebrojavanja rezultata, kroz koji se povezuju svi prethodno opisani mehanizmi:

1. Administrator u kartici “Izbori i kandidati” stvara novi izbor i dodaje kandidate.
2. Administrator otvara izbor. Poslužitelj tom prilikom generira par RSA ključeva (poglavlje 4); javni ključ sprema se uz izbor, a privatni se prikazuje administratoru točno jednom - taj trenutak je jedina prilika da ga sačuva izvan sustava.
3. Za svako biračko mjesto koje sudjeluje registrira se uređaj (dobiva se API ključ uređaja, odjeljak 6.1) i generira skup jednokratnih tokena (odjeljak 4.4) u količini koja odgovara broju birača.
4. Birač dolazi na glasačko mjesto, izbornu povjerenstvo tog mjesta provjerava dokumente glasača u nepovezanoj bazi, a zatim mu nasumično izdaje token. Glasac tada u terminalu unosi token, bira kandidata i potvrđuje glas. Poslužitelj istovremeno provjerava i troši token, šifrira glas hibridnom shemom (poglavlje 4), dodaje ga u lanac integriteta (poglavlje 5) i emitira stanje administratorskoj konzoli (poglavlje 3).

5. Nakon što je glasanje završeno, administrator zatvara izbor.
6. U kartici "Nadzor", administrator unosi ranije sačuvani privatni ključ. Sustav provjerava cijeli lanac integriteta, dešifrira sve glasove i prikazuje rezultate po kandidatu, po NUTS2 regiji i po županiji.

7 | Zaključak

U ovom radu razvijen je sustav za elektroničko glasanje čiji je glavni cilj komunikacija u stvarnom vremenu putem WebSocket protokola - poslužiteljska aplikacija i administratorska konzola povezane su trajnom, dvosmjernom vezom koja administratoru omogućuje praćenje tijeka glasanja uživo, bez ručnog osvježavanja i bez izlaganja osjetljivih podataka, uz mehanizam održavanja veze (ping/pong) koji jamči pouzdanost te veze tijekom cijelog trajanja izbora.

Uz to, glavni sigurnosni zahtjevi sustava - tajnost glasa, integritet rezultata i otpornost na dvostruko glasanje - riješeni su na razini arhitekture, a ne samo organizacijskim pravilima izbora, čime se eliminirala povezanost između identiteta birača i rezultata. Enkripcija osigurava da rezultati postoje samo u šifriranom obliku sve do zatvaranja izbora i unosa privatnog ključa; jedinstvena potrošnja tokena na razini baze podataka sprječava dvostruko glasanje čak i pod stvarnim paralelnim opterećenjem. Lančani hash omogućuje otkrivanje naknadne izmjene podataka bez potrebe za njihovim dešifriranjem baš poput *blockchain* tehnologije, a revizijski trag daje administratoru uvid u sigurnosno važne događaje bez izlaganja osjetljivih podataka.

Sustav je dizajniran s ciljem buduće primjene u stvarnim izbornim procesima, uz moguće prilagodbe specifičnim zakonskim i tehničkim zahtjevima nadležnih tijela.

Literatura

- [1] FASTAPI, *FastAPI documentation*. Dostupno na <https://fastapi.tiangolo.com/> (svibanj 2026.).
- [2] SQLALCHEMY, *SQLAlchemy 2.0 Documentation*. Dostupno na <https://docs.sqlalchemy.org/> (svibanj 2026.).
- [3] REACT, *React Documentation*. Dostupno na <https://react.dev/> (svibanj 2026.).
- [4] I. FETTE, A. MELNIKOV, *RFC 6455 - The WebSocket Protocol*, Internet Engineering Task Force, 2011.
- [5] M. JONES, J. BRADLEY, N. SAKIMURA, *RFC 7519 - JSON Web Token (JWT)*, Internet Engineering Task Force, 2015.
- [6] K. MORIARTY I DR., *RFC 8017 - PKCS #1: RSA Cryptography Specifications Version 2.2*, Internet Engineering Task Force, 2016.
- [7] M. DWORKIN, *NIST Special Publication 800-38D - Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, National Institute of Standards and Technology, 2007.
- [8] A. BIRYUKOV, D. DINU, D. KHOVRATOVICH, S. JOSEFSSON, *RFC 9106 - Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*, Internet Engineering Task Force, 2021.
- [9] PYTHON CRYPTOGRAPHIC AUTHORITY, *pyca/cryptography documentation*. Dostupno na <https://cryptography.io/> (svibanj 2026.).

Sažetak

U ovom radu opisan je razvoj sustava za elektroničko glasanje koji se sastoji od poslužiteljske aplikacije (FastAPI, PostgreSQL), administratorske konzole i glasačkog terminala (React), testiranog na podacima 129 stvarnih hrvatskih biračkih mjesta. Rad se fokusira na implementaciju komunikacije u stvarnom vremenu putem WebSocket protokola, kojom administratorska konzola dobiva trenutačan uvid u tijek glasanja bez potrebe za ručnim osvježavanjem podataka, uz opis dva odvojena komunikacijska kanala i mehanizma održavanja veze (ping/pong). Uz to, opisani su i sigurnosni mehanizmi koji štite podatke koji se tom komunikacijom razmjenjuju. Ti mehanizmi obuhvaćaju model u kojem se privatni ključ izbora prikazuje administratoru samo jednom i nikad ne pohranjuje, jedinstvenu potrošnju jednokratnih tokena na razini baze podataka koja sprječava dvostruko glasanje čak i pod paralelnim opterećenjem, lančani integritetski hash koji omogućuje otkrivanje naknadne izmjene podataka, te revizijski trag sigurnosno relevantnih događaja.

Ključne riječi

elektroničko glasanje, WebSocket, komunikacija u stvarnom vremenu, RSA, hibridna enkripcija, integritet podataka, revizijski trag, FastAPI, React

Electronic Voting System

Summary

This thesis describes the development of an electronic voting system consisting of a server application (FastAPI, PostgreSQL), an administrator console and a voting terminal (React), tested on data from 129 real Croatian polling stations. The thesis focuses on the implementation of real-time communication via the WebSocket protocol, through which the administrator console receives an immediate view of the voting process without manually refreshing data, including two separate communication channels and a connection keep-alive mechanism (ping/pong). In addition, the thesis describes the security mechanisms that protect the data exchanged through this communication. These mechanisms include a model in which the election's private key is shown to the administrator only once and never stored, unique consumption of single-use voting tokens at the database level that prevents double voting even under concurrent load, a chained integrity hash that enables detection of subsequent data tampering, and an audit trail of security-relevant events.

Keywords

electronic voting, WebSocket, real-time communication, RSA, hybrid encryption, data integrity, audit trail, FastAPI, React

Životopis

Zovem se Ivan Vuletić i rođen sam 27. siječnja 2004. u Osijeku. Osnovnoškolsko obrazovanje završio sam u Osnovnoj školi "Retfala", dok srednjoškolsko u III. gimnaziji u Osijeku. 2022. godine upisujem preddiplomski sveučilišni studij Matematike i računarstva na Fakultetu primijenjene matematike i informatike Sveučilišta J. J. Strossmayera u Osijeku